Supplementary Methods

Semi-Automatic, Annotation-Guided Mapping
from Visual DSD to DMFB Assay Graphs

Abstract

This supplement specifies the graph model, annotation contract, parsing stages, interactive editing workflow, and initial-DAG export used by DSD Assay Lite. It separates molecular program structure from physical preparation decisions and explains where user input is required. A complete sample–buffer example and validation scenarios support interpretation and reproduction of the mapping. The described interface is a partial translator, not a full Visual DSD compiler, biochemical verifier, or DMFB scheduler.

Contents

S1 Scope and User Responsibilities **3**

S2 Formal Definition of the Assay Graph **5**

S2.1 Structural and Temporal Constraints 6

S3 Dilution through Mixing **6**

S4 Protocol Annotation Contract **6**

S4.1 Node and Edge Construction 7

S4.2 Reference Resolution and Chained Operations 7

S5 Lexical and Syntactic Analysis **7**

S6 Conversion and Initial-DAG Export **8**

S7 Graphical Editing and Round-Trip Consistency **9**

S7.1 Application of an Edited Draft 9

S7.2 Display Metadata 10

S8 Complete Sample–Buffer Example **10**

S8.1 Explicit Branching 12

S9 Initial-DAG Export Contract **12**

S10 Validation Scenarios and Reproducibility **12**

S11 Limitations and Interpretation **13**

S1 Scope and User Responsibilities

Visual DSD describes and analyses DNA strand-displacement systems [1, 2]. Our interface adds a protocol annotation layer for preparing a DMFB assay graph. Native molecular descriptions do not, by themselves, identify physical reservoirs, specify heating or detection hardware operations, or determine a droplet-splitting protocol. This does not imply that Visual DSD cannot model temperature effects, reporters, or other related molecular behaviour.

The mapping is semi-automatic: the software extracts program structure, expands supported module calls, builds dependencies, and checks the resulting graph; the user supplies or confirms physical operations, parameter values, execution dependencies, and port assignments. A reporter species is not automatically a DETECT operation. It may be an independently dispensed reagent, while detection is a separate readout operation that must be specified by the protocol.

Each supported independent executable block is treated as a separate preparation. Program definitions are a symbol environment, not a list of physical inputs. An initial molecular population is also not a droplet count. The current editing model binds each positive extracted occurrence to one dispense node; this is an interface convention, not a claim that a physical droplet contains only one molecular species. Preparing premixed inputs requires an explicitly agreed input representation.

Graphical editing is intended to reduce repetitive annotation work and make dependencies easier to inspect. No measured reduction in preparation time, user errors, or biochemical failure rate is claimed here.

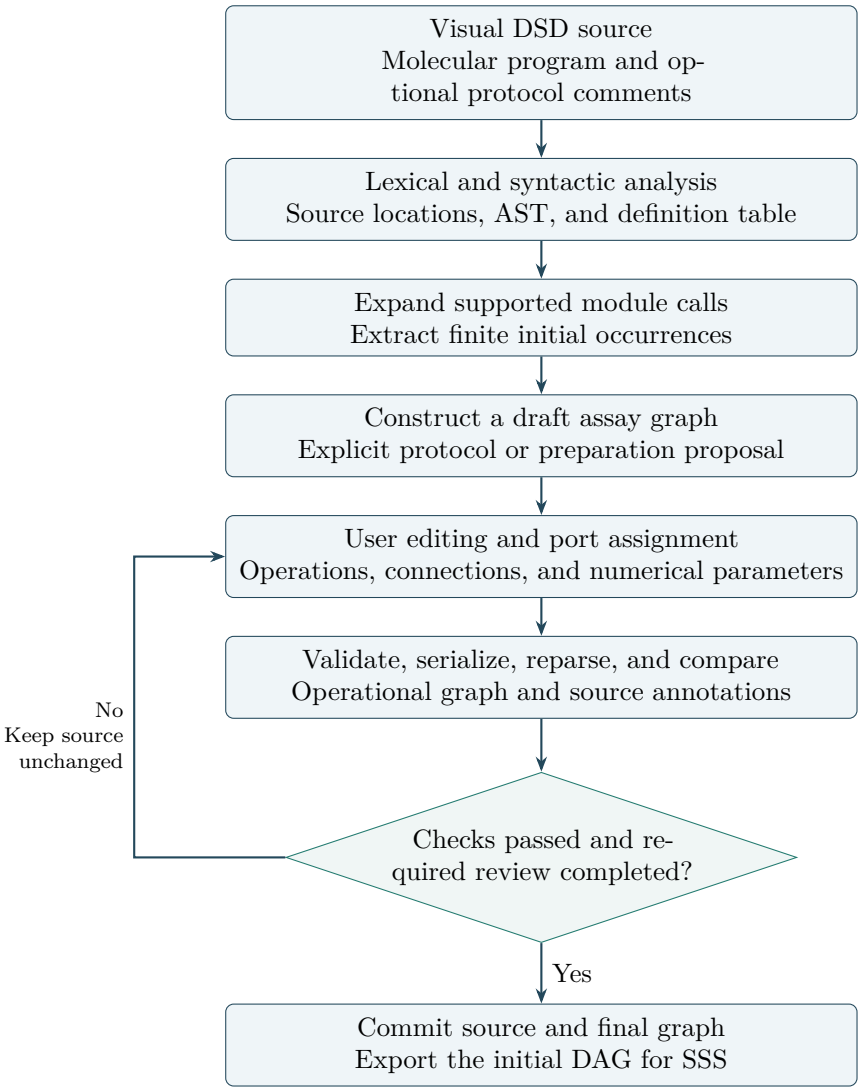


Figure S1: Conceptual workflow of semi-automatic graph construction and application of graphical edits. Export remains conditional on validation and review. Arrows indicate processing flow, not droplet movement or execution time.

S2 Formal Definition of the Assay Graph

An assay is represented as a typed directed acyclic graph. Each node denotes an operation occurrence; separate occurrences remain distinct even when they have the same operation type or involve the same molecular species. Each edge links an operation to a consumer of its product and therefore imposes a precedence constraint. Under the adopted preparation model, independent executable blocks yield separate graphs.

We define

$$\begin{aligned} &G = (V, E, \tau, \alpha, \pi, \rho), \\ &E \subseteq V \times V, \qquad V \neq \varnothing, \end{aligned} \tag{S1}$$

where V contains operation occurrences and E contains their directed dependencies. The function τ assigns operation types, α records applicable numerical attributes, π assigns input and output ports, and ρ retains provenance, including molecular species, source locations, and protocol identifiers. Display coordinates are stored separately and do not affect the assay specification.

The type function is

$$\tau : V \longrightarrow T, \tag{S2}$$

where

$$\begin{aligned} T = \{&\text{DISPENSE}, \text{MIX}, \text{SPLIT}, \\ &\text{HEAT}, \text{DETECT}, \text{OUTPUT}\}. \end{aligned} \tag{S3}$$

Dilution is represented as mixing the sample with water supplied through a designated input port, rather than as a separate operation type.

For the target architecture, the available port names are

$$\begin{aligned} I = \{&\texttt{i1}, \texttt{i2}, \texttt{i3}, \texttt{i4}, \texttt{i5}, \\ &\texttt{t1}, \texttt{t2}, \texttt{t3}, \texttt{t4}, \texttt{t5}\}, \end{aligned} \tag{S4}$$

$$\begin{aligned} O = \{&\texttt{output1}, \texttt{output2}, \texttt{output3}, \\ &\texttt{output4}, \texttt{output5}\}. \end{aligned} \tag{S5}$$

For $q \in T$, let

$$V_q = \{v \in V \mid \tau(v) = q\}. \tag{S6}$$

Thus, V_{DISPENSE} and V_{OUTPUT} contain the input and output operations, respectively. The port assignment consists of

$$\begin{aligned} &\pi_i : V_{\text{DISPENSE}} \longrightarrow I, \\ &\pi_o : V_{\text{OUTPUT}} \longrightarrow O. \end{aligned} \tag{S7}$$

Node identifiers and port names serve different purposes and need not coincide.

Table S1 summarizes the structural constraints and numerical fields under the adopted SSS mapping. Here, $d^-(v)$ and $d^+(v)$ denote the in-degree and out-degree. The interpretations of `value` are mapping conventions reported in the source methodology, not an independently verified specification of every SSS version; units must be specified consistently with the target SSS interface and are not inferred from Visual DSD population values.

A binary split requires two distinct successors but does not specify their volume ratio. The detection parameter defaults to `count`$= 1$; it is not the in-degree of a detection node. These constraints establish structural consistency; they do not guarantee biochemical validity or physical executability on a DMFB.

The numerical interpretations in Table S1 follow the adopted target-interface convention. The supplied source material does not establish physical units or version-specific SSS implementation details. These

Table S1: Degree constraints and numerical fields under the adopted mapping.

Operation	d^-	d^+	Numerical fields
DISPENSE	0	1	`value`: volume
MIX	$k \geq 2$	1	`count`= k; `value`: mixing duration
SPLIT	1	2	`count`= 2; `value`: splitting duration
HEAT	1	1	`value`: heating duration
DETECT	1	1	`count`≥ 1 (integer); `value`: detection duration
OUTPUT	1	0	None

must be confirmed for the deployed SSS interface before executable experimental parameters are assigned. In particular, a heating value is not a temperature setpoint under the duration convention.

S2.1 Structural and Temporal Constraints

Self-loops, duplicate edges, unresolved references, and cycles are rejected. Every non-output node other than SPLIT has exactly one consumer. Two downstream consumers require an explicit split; a reference cannot silently duplicate a droplet. These constraints do not prove volume conservation.

If scheduling is performed downstream, an edge (u, v) imposes the necessary precedence condition

$$t(v) \geq t(u) + d(u), \tag{S8}$$

where t is the start time and d the operation duration. With a separately modelled transfer delay $\delta(u, v)$, the right-hand side becomes $t(u) + d(u) + \delta(u, v)$. The interface does not compute these times, and identifying `value` with d requires an explicit target-interface contract.

S3 Dilution through Mixing

Dilution is represented by mixing the sample with a diluent, such as water, supplied through a designated input port. The sample and diluent are connected to a MIX node; therefore, no separate DILUTE operation is included in the assay graph.

S4 Protocol Annotation Contract

Operational annotations use the comment channel and are defined by this interface, not by the native Visual DSD grammar. Standalone examples below use `//`; other comments remain separate from executable tokens. Annotation names are case-sensitive: `@Heat` is not `@heat`.

Protocol identifiers begin with an ASCII letter or underscore and continue with letters, digits, or underscores. Reference lists use commas without spaces, for example `inputs=A,B,C`. Numerical `value` fields are optional and default to 1. Values must be finite and positive; counts must be positive integers. MIX count is determined by the number of distinct inputs, SPLIT count is fixed at 2, and DETECT count defaults to 1. A literal `assay` prefix is not required.

S4.1 Node and Edge Construction

Table S2 gives independent translations. For the first six rows, use $A \mapsto 1$, $B \mapsto 2$, $M \mapsto 3$, $S \mapsto 4$, $H \mapsto 5$, $D \mapsto 6$, and OUT1 $\mapsto 7$. The final two rows state their own identifier assumptions. These fragments must not be concatenated into a single complete assay: connecting M simultaneously to SPLIT and HEAT would violate the single-consumer rule.

Table S2: Independent annotation-to-DAG translation examples.

Annotation	Graph encoding and interpretation
`@node id=A fluid=i1 value=1`	`NODE(1,DISPENSE,i1,1,i1)` Bind the input occurrence to port `i1`.

`@mix id=M inputs=A,B value=4`	`NODE(3,MIX,2,4,M)`
	`EDGE(1,3)`; `EDGE(2,3)`
	Input references determine the MIX count.

`@split id=S input=M value=1`	`NODE(4,SPLIT,2,1,S)`
	`EDGE(3,4)`
	Two distinct consumers must be supplied later.

`@heat id=H input=M value=30`	`NODE(5,HEAT,30,H)` `EDGE(3,5)`
`@detect id=D input=H count=1 value=5`	`NODE(6,DETECT,1,5,D)` `EDGE(5,6)`
`@output id=OUT1 input=D output=output1`	`NODE(7,OUTPUT,output1,OUT1)` `EDGE(6,7)`

`@heat value=30 @detect value=5`	`NODE(4,HEAT,30,M_HEAT_4)`
	`NODE(5,DETECT,1,5,M_DETECT_5)`
	`EDGE(3,4)`; `EDGE(4,5)`
	Assumes attachment to node M numbered 3 and subsequent IDs 4 and 5.

`@node output=output1`	`NODE(7,OUTPUT,output1,OUT_7)`
	`EDGE(6,7)`
	Assumes the group's terminal node is 6 and the new output ID is 7.

S4.2 Reference Resolution and Chained Operations

The graph builder resolves references through a protocol-identifier table. Appended operations are connected in textual order. After a MIX annotation followed by inline HEAT and DETECT annotations, subsequent references to the MIX identifier consume the terminal DETECT result, not an additional copy of the mixture. Explicit standalone identifiers are preferable when the intended dependency needs to be unambiguous.

The terminal output annotation on a group connects an OUTPUT node to the group's final operation. Port names must belong to the architecture-specific sets in Eqs. S4 and S5; the same textual protocol identifier is not a substitute for a port assignment.

S5 Lexical and Syntactic Analysis

Lexical analysis identifies tokens and comments while preserving source locations for diagnostics. Line and block comments are separated from executable tokens, but protocol annotations are retained. A recursive descent parser then identifies declarations, module definitions, and executable blocks. A species

Algorithm S1: Lexical and syntactic analysis

Input : Visual DSD source text S
Output : A, D, H, or source-located diagnostics
Initialize $\mathcal{L}, D, H, A$
while *unconsumed source text remains* **do**
    Read the next token or comment and its source location
    if *the item is a comment* **then**
        Preserve protocol annotations and their locations in H
    else
        Append the token to $\mathcal{L}$
Parse $\mathcal{L}$ using recursive descent
Record declarations in A and module definitions in D
Represent executable blocks in A
Retain supported molecular expressions and check delimiter matching
return (A, D, H)
`// On any lexical or syntactic error:  report its location and terminate with failure.`

reference in the `plots` directive is not treated as a material input, and a `def` declaration alone does not create an assay node.

Algorithm S1 returns the abstract syntax tree A, module definition table D, and annotation collection H. The local symbol $\mathcal{L}$ denotes the token sequence and is distinct from the operation-type set T.

Within the supported subset, molecular expressions are retained without interpreting DNA strand-displacement semantics. Delimiter matching is a structural check, not a complete validation of molecular syntax or semantics. Annotation placement is checked during subsequent semantic analysis.

S6 Conversion and Initial-DAG Export

Algorithm S2 consumes the outputs of Algorithm S1; it does not scan or parse the source again. Relevant declarations and executable blocks are retrieved from A, and invoked modules are expanded using D. Only initial occurrences with finite positive quantities are considered candidates for material inputs; molecular population values are not interpreted as droplet counts. Zero-valued occurrences may be retained as model metadata.

Protocol annotations supply physical instructions. For example,

```
// @heat id=H input=M value=30
```

introduces a heating node with identifier `H`, connects it to the output of `M`, and assigns its numerical parameter. These annotations are defined by our interface and carried in Visual DSD comments; they are not native Visual DSD operation declarations.

With explicit protocol commands, graph construction follows the annotated operations and dependencies. Otherwise, the interface proposes a preparation graph for review. Interpreting parallel composition as physical mixing is a preparation assumption, not a direct consequence of the Visual DSD operator `|`. Introduced assumptions and proposed port assignments must be presented to the user and confirmed before export.

Validation checks references, operation types, numerical parameters, port assignments, node degrees, and acyclicity. Blocking errors prevent export regardless of user confirmation. The accepted graph is encoded as `NODE` and `EDGE` records in the initial-DAG format. Scheduling, resource allocation, and droplet routing remain downstream tasks.

Algorithm S2: Assay graph construction and export

Input : A, D, H from Algorithm S1
Precondition : Lexical and syntactic analysis succeeded
Output : Per-block graphs, inventories, diagnostics, and review flags
Retrieve declarations and executable blocks from A
if *no executable block exists* **then**
  Report a missing-block error and return failure
Initialize result collection R
foreach *executable block* b **do**
  Expand invoked modules using D and declarations
  Extract finite initial occurrences into inventory J
  Validate and bind the relevant annotations from H
  `// On a blocking error above, record failure for` b `and skip its remaining steps.`
  if *explicit protocol commands exist* **then**
    Build G by resolving annotated dependencies
  else
    Propose preparation graph G
  Collect construction diagnostics and introduced assumptions
  if G *exists* **then**
    Check references, types, parameters, ports, degrees, and acyclicity
  Set r to true if assumptions were introduced
  Append $(G, J, \text{diagnostics}, r)$ to R
return R

Export a selected graph G**:**
Require G to exist with $V(G) \neq \emptyset$
Require no blocking diagnostics
Require user confirmation if its review flag is true
Require final validation of G to succeed
return the initial-DAG encoding of G

S7 Graphical Editing and Round-Trip Consistency

The editor presents extracted input occurrences with their Visual DSD names, such as `Sample()`, so that users can identify the relevant material before assigning ports. The final graph emphasizes assigned architecture ports instead. Users can insert or remove operations, change types and applicable parameters, connect nodes, reposition them, and undo or redo draft edits. Adding a genuinely new material requires a corresponding source occurrence; an unbound graphical dispense node cannot silently invent a reagent.

S7.1 Application of an Edited Draft

Let S be the source used to create the draft, G^* the edited graph, W the source rewriter, and P the graph-construction pipeline. Application first produces a candidate source and reparses it:

$$S^* = W(S, G^*), \qquad G' = P(S^*). \tag{S9}$$

Let Π retain protocol identifiers, operation types, numerical parameters, port assignments, and edges, with internal numeric identifiers normalized through protocol identifiers. The acceptance condition is

$$\Pi(G') = \Pi(G^*). \tag{S10}$$

This checks consistency of the operational representation, not equality of source text or molecular behaviour. Formatting may change. A common defect shared by the writer and parser can escape this check, so round-trip consistency is not a substitute for independent test cases.

The selected block is rewritten using canonical annotations, while source outside that block remains intact. Relevant retained metadata and non-protocol comments are carried forward. Before application, the editor checks that the source has not changed since the draft was loaded. If validation or reparsing fails, the candidate is not committed; the user corrects the draft or reloads the current source.

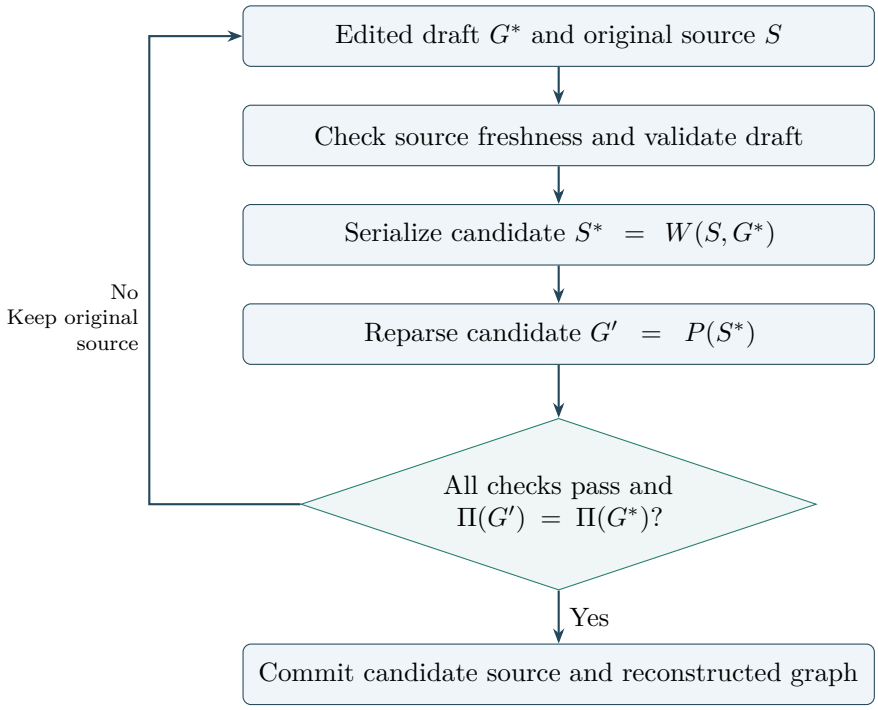


Figure S2: Transactional application of graphical edits. An error at any preceding check rejects the candidate; successful serialization alone is insufficient for acceptance.

S7.2 Display Metadata

Layout is recorded separately, for example:

```
// DSD-GRAPH-LAYOUT {"A":[30,45],"B":[220,45],"M":[125,169]}
```

Listing S1: Example of optional display metadata.

These coordinates position nodes in the editor. They do not describe electrode locations, droplet routes, or execution times. The implementation rounds coordinates during serialization and checks the rounded positions after reparsing. Any implementation-specific coordinate bounds are editor limits, not part of the physical graph model.

S8 Complete Sample–Buffer Example

Listing S2 specifies two initial occurrences and a complete physical protocol. The initial population of each species is 10. These values are molecular-model data, independent of the operation fields 1, 4, 30, and 5. The chosen operations are mixing, heating, detection, and output; their order comes from the annotations rather than from the molecular structures. The output protocol identifier is written as `OUT1` to avoid confusion with the port-name set O.

```
def Sample() = <t^ sample>
def Buffer() = <t^ buffer>
(
  10 Sample() // @node id=A fluid=i1 value=1
| 10 Buffer() // @node id=B fluid=i2 value=1
// @mix id=M inputs=A,B value=4
// @heat id=H input=M value=30
// @detect id=D input=H count=1 value=5
// @output id=OUT1 input=D output=output1
)
```

Listing S2: Annotated Visual DSD source for the worked example.

The graph has six nodes and five edges:

$$E = \{(A, M), (B, M), (M, H), (H, D), (D, \mathrm{OUT1})\}. \tag{S11}$$

Figure S3 shows the dependency graph. The code in Listing S3 uses numerical IDs for the same nodes. Species names, protocol identifiers, numeric graph IDs, and architecture port names belong to different naming layers and should not be conflated.

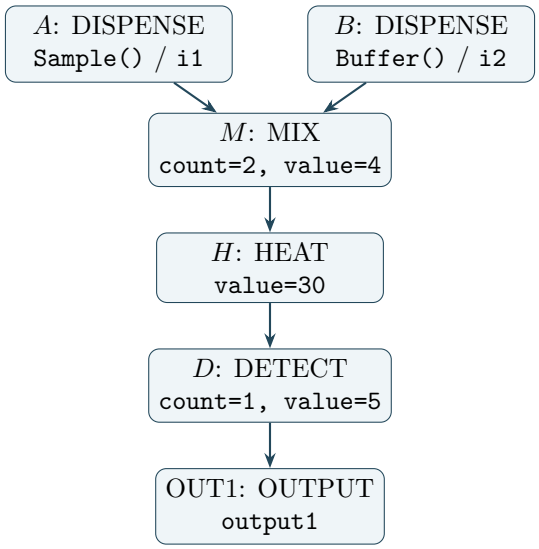


Figure S3: Dependency graph for Listing S2. This vector diagram is a schematic representation of the expected graph, not a screenshot or a physical placement.

```
DAGNAME(DSD_Assay)
NODE(1,DISPENSE,i1,1,i1)
NODE(2,DISPENSE,i2,1,i2)
NODE(3,MIX,2,4,M)
NODE(4,HEAT,30,H)
NODE(5,DETECT,1,5,D)
NODE(6,OUTPUT,output1,OUT1)
EDGE(1,3)
EDGE(2,3)
EDGE(3,4)
EDGE(4,5)
EDGE(5,6)
```

Listing S3: Initial-DAG encoding of the worked example.

S8.1 Explicit Branching

To send the result of a single operation to two consumers, insert a split. Listing S4 is an alternative continuation after the MIX node in Listing S2; it replaces the heating, detection, and output continuation rather than being appended to it.

```
// @split id=S input=M value=1
// @output id=OUT1 input=S output=output1
// @output id=OUT2 input=S output=output2
```

Listing S4: Alternative split continuation with two distinct consumers.

This produces the edges (M, S), $(S, \mathrm{OUT1})$, and $(S, \mathrm{OUT2})$. Connecting both a split and another operation directly to M would be invalid in this graph model.

S9 Initial-DAG Export Contract

The exported syntax follows the interface's initial-DAG encoder. A file starts with `DAGNAME`, followed by node and edge records. Table S3 lists the field order. Names shown in italic are placeholders, not literal tokens. Each node ID must be unique and every edge endpoint must exist. IDs encode identity, not

Table S3: Record templates used by the initial-DAG encoder.

Type	Record template
DISPENSE	`NODE(`*id*`,DISPENSE,`*fluid*`,`*value*`,`*name*`)`
MIX	`NODE(`*id*`,MIX,`*count*`,`*value*`,`*name*`)`
SPLIT	`NODE(`*id*`,SPLIT,2,`*value*`,`*name*`)`
HEAT	`NODE(`*id*`,HEAT,`*value*`,`*name*`)`
DETECT	`NODE(`*id*`,DETECT,`*count*`,`*value*`,`*name*`)`
OUTPUT	`NODE(`*id*`,OUTPUT,`*port*`,`*name*`)`
Dependency	`EDGE(`*source-id*`,`*target-id*`)`

chronological execution order. The final name field is a graph label, whereas the fluid or output field selects a port. Automatic node numbering may change when the graph is edited without changing its dependencies. The example demonstrates the encoder's output; it is not evidence that a specific external SSS build or a physical device has executed the assay.

S10 Validation Scenarios and Reproducibility

Table S4 distinguishes expected acceptance and rejection cases. These are reproducible structural checks, not measured biochemical or hardware outcomes. A regression test should inspect node types, normalized identifiers, ports, parameters, and edges rather than merely comparing an image.

For reproduction, load Listing S2, inspect the source-bound input nodes, verify the port assignments, and generate the final graph. Compare the result with Eq. S11 and Listing S3. Then edit one operation value, apply the change, and confirm both the source annotation and reconstructed graph agree. A separate negative test should deliberately create an invalid branch and verify that export is blocked.

Record the exact interface release or commit, the selected main block, all annotations, target architecture, numerical units, and the SSS version when reporting experiments. Retain the annotated source and exported DAG together. Structural test success, biochemical fidelity, scheduling feasibility, and measured

Table S4: Representative validation scenarios and expected outcomes.

Scenario	Expected result
Sample–buffer protocol	Six nodes and five edges; all required ports assigned.
Three-input MIX	Accept when three distinct predecessors exist and the count equals three.
Two consumers of a MIX	Reject unless an explicit SPLIT mediates the branch.
SPLIT with one consumer	Reject: two distinct successors are required.
Duplicate protocol ID	Reject ambiguous identity.
Unknown reference or port	Reject unresolved dependency or invalid port assignment.
Cycle or self-loop	Reject; an assay DAG must be acyclic.
Nonpositive/nonfinite value	Reject; numerical operation fields must satisfy the contract.
Noninteger DETECT count	Reject; the count must be a positive integer.
Unconfirmed proposal	Block export while required user review is incomplete.
Source modified after draft load	Reject application and require reloading the draft.
Round-trip parameter/edge mismatch	Reject candidate source and preserve the original.

user effort are separate evaluation questions and should be reported separately. This supplement does not supply a new timing benchmark or an execution-accuracy measurement.

S11 Limitations and Interpretation

The parser handles a supported subset rather than the complete Visual DSD language. The algorithms describe the pipeline and must not be read as a promise to accept arbitrary domain scoping, parameter metadata, sweeps, spatial models, time-dependent populations, or unbounded expansion. Unsupported constructs must be diagnosed or explicitly excluded from export; omitted molecular detail must not silently become a fabricated physical protocol.

A graph can be structurally valid while having unsuitable volumes, temperatures, detection settings, reservoir contents, or timings. These require experimental knowledge and downstream checks. Reservoir mapping does not prove that the physical port contains the intended reagent. Similarly, interpreting a named Reporter as an optical detector would confuse a molecular reagent with a hardware operation.

The preparation-proposal path does not infer molecular reaction order. It provides an editable starting point under declared assumptions. The user remains responsible for whether that preparation matches the intended experiment. The round-trip check verifies representation consistency but cannot establish biochemical equivalence, complete Visual DSD semantics, or manufacturability of the assay.

The architecture configuration used in an associated study should be described separately from this mapping procedure. Neither the notation nor the editor establishes an optimal architecture. Scheduling, placement, routing, capacity checks, and physical validation belong to the subsequent synthesis and execution workflow.

References

[1] C. Spaccasassi, N. Dalchau, M. Lakin, R. Petersen, and A. Phillips, *Visual DSD User Manual*, March 3, 2019. https://www.microsoft.com/en-us/research/wp-content/uploads/2009/02/Visual_DSD_Manual.pdf.

[2] M. R. Lakin, S. Youssef, F. Polo, S. Emmott, and A. Phillips, "Visual DSD: a design and analysis tool for DNA strand displacement systems," *Bioinformatics*, vol. 27, no. 22, pp. 3211–3213, 2011. https://doi.org/10.1093/bioinformatics/btr543.